

Tourmaline Core Junior Test Assignment – Change Calculation (1st Iteration)

You need to implement the logic of change calculation. All needed cases should be covered by unit tests and the solution has to be developed according to TDD (Test-Driven Development).

Let's imagine you are a salesman, and you need to return the change to a buyer or tell this person that you don't have it and the payment can be done only using a credit card. For simplicity you have only the following possible nominal values: 100, 50, 10, 5, 2, and 1. You might not have all nominal values at your checkout point, e.g. you don't have coins with the nominal value 5. At the same time, you might have several coins with the same nominal value, e.g. 25 coins with the nominal value 1.

Implement a function that can tell if the needed amount of change can be returned to a buyer and if it can also return the list of coins that will make it.

Example

For instance, you have the following coins 5, 3, and 1. If I ask you about the change that equals to 4 you can return it and the coins will be 3 and 1. However, you cannot return the change that equals to 2.

Non-Functional Requirements

- Use any unit testing library.
- Follow TDD. That is one of the key practices that we as a tech company follow.
- Create a public GitHub repository to share your solution with us. We like to open-source everything that we can to contribute to the tech community efforts. GitHub is what we prefer in most cases for open-source projects.
- Create an open pull-request with all code changes. It is the most convenient way for us to review your code and leave review notes.
- Create a GitHub Actions pipeline. It should run all unit tests on each commit only in open pull-requests. If the pipeline fails it shouldn't be allowed to merge the pull-request. That is how we automate the tests execution in all our projects.
- The atomic commits approach should be used. Thus, you commit each piece of your progress step by step, e.g. adding a test which fails (commit), writing the minimal amount of code for it to work (commit), refactor if necessary (commit), etc. This compliments TDD in a way that it is maximally transparent for a reviewer and shows how your solution evolves step by step.
- Stick to Conventional Commits approach to formulate your commit messages. The text of the commits should be clean, verbose, and helpful. Conventional Commits is one of the widely spread industry standards that we follow.

- The code should be formatted in a uniform style, there should be no unused usings, variables, etc. Make it look clean and production ready that we enjoy reviewing it :)

How do you know if you failed the assignment?

It is automatically not accepted and won't be checked if you cannot create an open pull-request for review or there are no atomic commits or TDD wasn't followed.

All bits of functional and non-functional requirements are important and need to be followed. We use this assignment not only to check your tech skills when you receive this task but also to see what you can learn down the road.

Please don't hesitate to ask any kind of question. What you ask could tell us a lot about the way you think. Please don't be afraid to ask anything, we were in your position back then and won't judge you. Good luck!